

Class 6: R functions

Samarah Zaman (PID: A17411141)

Table of contents

Background	1
A first function	1
A <code>generate_dna()</code> function	3
Write a <code>generate_protein()</code> function	5
Connecting our findings to immunology	7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *argument* (one or more comma seperated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either “**required**” or “**optional**”. The later have fall-back default values that will be used if the user does not specify them.

Q1a: Your first version of the function should add two input numbers together.
For example, `add(4, 7)` should return 11. [1 pt]

```
add<- function(x, y=1000) {  
  x + y  
}
```

Can we use our new function:

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 1010
```

```
add(4, 7)
```

```
[1] 11
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

```
add <- function(x, y=0){  
  sum(x,y)  
}
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`. [2 pts]

```
add<- function(...) {  
  sum(...)  
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using `return()` function call.

```
add<- function(...) {
  return(sum(x,y,...))
  cat("Is it break time yet?\n")
}
```

A generate_dna() function

A useful function here is the base “R” `sample()` function:

```
sample(1:5, size=60, replace=TRUE)
```

```
[1] 3 2 1 1 5 2 3 3 3 1 1 2 1 3 2 2 3 4 3 2 4 4 1 4 3 5 5 5 3 3 5 3 1 5 1 1 1 2
[39] 1 2 3 1 4 5 1 1 2 2 4 4 4 3 4 2 2 2 4 3 5 4
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A","C","G","T"), size=10, replace=TRUE)
```

```
[1] "A" "T" "C" "G" "C" "A" "C" "T" "G" "T"
```

Q2a: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna <- function(len){
  sample(x=c("A","C","G","T"), size=len , replace=TRUE)}
```

```
generate_dna(6)
```

```
[1] "C" "G" "G" "C" "G" "A"
```

Q2b: Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
generate_dna <- function(len, single.element=TRUE){
  ans<- sample(x=c("A","C","G","T"), size=len , replace=TRUE)
  if(single.element){
    ans<- paste(ans, collapse="")
  }
  return(ans)
}
```

```
generate_dna()
```

```
[1] "CAAA"
```

```
generate_dna(single.element= FALSE)
```

```
[1] "A" "A" "A" "G"
```

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: len9 CGAAGGCTG Be sure to include clear comments that explain each step of your function. [2 pts]

```
generate_dna <- function(len, single.element = TRUE) {
  # sample 'len' bases from A, C, G, T with replacement
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)

  # if single.element TRUE, collapse the vector into one continuous string
  if (single.element) {
    ans <- paste(ans, collapse = "")
  }

  # print FASTA id line showing sequence length
  cat(paste(">len", len, "\n", sep=""))

  # print the sequence itself, followed by a newline
  cat(ans, "\n")
}
```

```
generate_dna(9)
```

```
>len9
TTCGTTGCC
```

Write a generate_protein() function

Q3: Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return “WQRTAG”.

```
generate_protein <- function(len, single.element = TRUE) {  
  # Draw 'len' amino acids at random (20 standard amino acids), sampling with replacement  
  ansP <- sample(x = c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F",  
  # If single.element is TRUE, collapse the character vector into one contiguous string  
  # so the sequence is represented as a single string instead of a vector of single-letter c  
  if (single.element){  
    ansP <- paste(ansP, collapse = "")  
  }  
  
  # Print the sequence to the console  
  cat(ansP)  
}
```

```
generate_protein(6)
```

YGHREY

Q4: Generate random protein sequences of length 6 to 13. Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```
for(l in 6:13) {  
  cat(">id.", l, "\n", sep="")  
  cat(generate_protein(l), "\n")  
}
```

```
>id.6  
QTCTMG  
>id.7  
HVKFIQY  
>id.8  
TEDKDCIN
```

```

>id.9
GMNHRECSS
>id.10
RCCGNQWNAS
>id.11
SWQPRSNKIRH
>id.12
NIKDRIEDRSYD
>id.13
KYYERLLQPREEM

```

```
generate_protein(7)
```

WWWRDYH

Q5: BLASTp search against nr — are your peptides “unique in nature”? Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length	Identity	Coverage	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	77.78	100	Y
10	88.89	90	Y
11	100	73	Y
12	80	83	Y
13	83.33	77	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

The peptides first start to look “unique in nature” at length 9, where we no longer observe a 100% identity and 100% coverage match.

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^n for a peptide of length n) and to the size of the known protein universe. [1 pt]

Very short peptides are frequently found in nr because the total sequence space (20^L) is relatively small at low L , so many combinations already exist in the known protein universe. As peptide length increases, the sequence space grows exponentially, quickly far exceeding the number of sequences that have ever existed or been cataloged, making exact matches extremely unlikely.

Connecting our findings to immunology

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Based on the Q5 data, peptides begin to look “unique in nature” at length 9, so the minimum length for MHC class II presentation is likely around 9–12 amino acids. Very short peptides around 6–8 amino acids are not ideal because they frequently have exact matches in the nr database, meaning they are not specific and could correspond to many self-proteins. This occurs because the sequence space, 20^L is still relatively small at low L , so many sequences already exist within the known protein universe. As peptide length increases, the sequence space expands exponentially, making longer peptides much more unique. Presenting longer peptides therefore improves the immune system’s ability to distinguish self from non-self and reduces the risk of triggering autoimmune responses.